

Greed_MST

[04greedy-mst.pdf](#)

最小生成树

获取最小生成树的算法我们在数据结构阶段学过两种，分别是 *Prim* 和 *Kruskal* 算法，在这里我们再补充一种，*Reverse – Delete algorithm*。如果想要复习数据结构的图算法，推荐 B站 up主：蓝不过海呀，这是真神 🍻

- **Prim 算法：**

从某一个顶点开始，让一棵树一点点“长大”。每一步都从当前树的边界上，选一条权重最小的接到树外的新顶点。

- 任选一个起点 s
- 初始时树 T 只包含起点
- 每一步找一条边 e ，满足：
 - 它恰好一个端点在当前树 T 中
 - 并且它在这类边里权重最小
- 把这条边和新顶点加入树中
- 重复直到所有顶点都被加入

- **Kruskal 算法：**

从一开始什么边都没有，慢慢往里加边。每次优先考虑当前权重最小的边，但如果加入后会形成环，就不能加。

- 初始： $T = \emptyset$
- 将所有边按权重从小到大排序
- 依次检查每条边 e
 - 若加入 e 不会形成环，就把它加入 T
 - 若会形成环，就跳过
- 直到选出 $n - 1$ 条边为止

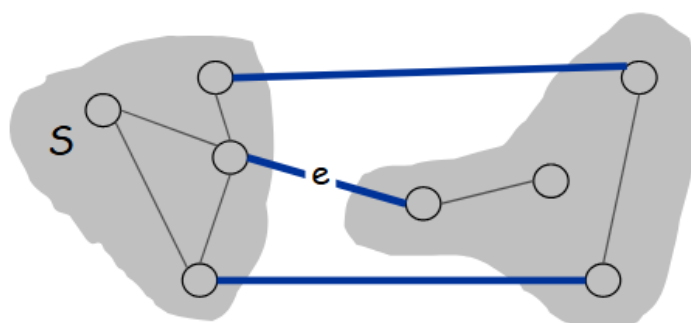
- **Reverse – Delete 算法：**

它和 Kruskal 有点“反着来”。不是从空集开始加边，而是先把所有边都放进去，然后不断删边。每次优先考虑当前权重最大的边，只要删掉它之后图仍然连通，就删掉它。

- 初始: $T = E$ (先把所有边都放进去)
- 将所有边按**权重从大到小排序**
- 依次检查每条边 e
 - 若删除 e **不会使图断开**, 就删除它
 - 若删了会断开, 就保留
- 最后剩下的就是 MST

简单证明 Prim 算法的最优性:

证明割边最优性:



e is in the MST

将连通图中的点任意分为两个部分, 这两部分至少由一条边连接, 割边就是所有连接边里最短的那一个。

割边最优性从直观上也很好理解, 假设存在一个MST, 它有一条边并不是割边: 那么我们就发现, 将这条边连接的两端分为两个子图, 然后将连接着两个子图的边替换为割边, 这样的操作一定不会使问题更差, 还可能会更好, **所以归纳地看, 每一步都选择割边的解是最优解 OPT。**

对任意一个点集 $S \subset V$, 如果一条边 e 是所有“恰好一个端点在 S 中”的边里**权值最小的**, 那么 e 一定属于某棵 MST。其实仍然使用到了交换论证的思想。

Prim 正确性:

设 Prim 算法依次选出的边 $e_1, e_2, \dots, e_{n-1}$, 其中 n 是图的顶点数。在第 k 步时, 设当前 Prim 已经选出的边形成的树覆盖的顶点集合为 S_k 。Prim 选择的下一条边 e_k 满足:

- e_k 恰好有一个端点在 S_k 中;
- 并且在所有这样的边中, e_k 权值最小。

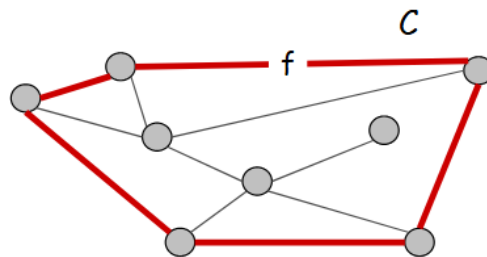
所以, e_k 就是割 $(S_k, V - S_k)$ 上的最小边。由**割性质**可知:

e_k 属于 MST

因为边权互异，MST 唯一，所以每一步 Prim 选出的边都属于这棵唯一的 MST。Prim 选出的每一条边都属于 MST，所以 Prim 得到的树就是 MST。

简单证明 Kruskal

证明环最大边的不存在性：Cycle property

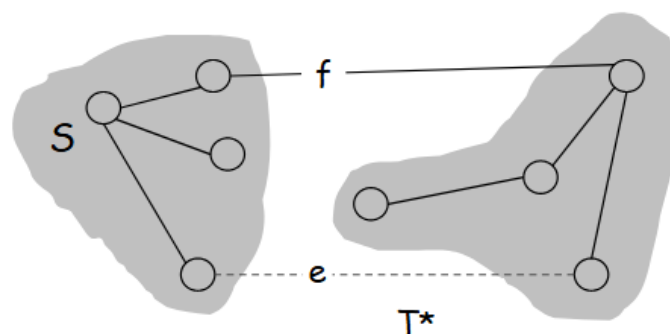


f is not in the MST

即如果 f 是环上最大权值的边，那么它一定不存在于 MST 中，其实从直觉上也很好发现，如果 f 是环上的一个边，我们完全可以取环的另一个边来替换 f ，由于 f 已经是最大权重，所以这样的操作并不会使情况更差。

假设存在一棵 MST，记为 T ，且 $f \in T$ 。由于 T 是生成树，所以删去边 f 后， T 会被分成两个连通分量，记其顶点集分别为 A 和 B 。设 f 的两个端点分别落在 A 和 B 中。

现在考虑原图中的环 C 。因为 f 在环 C 上，所以把 f 从环中去掉后，环上其余边仍构成一条连接 f 两端点的路径。因此，这条路径上必存在一条边 e ，它的一个端点在 A 中，另一个端点在 B 中。（看下图，既然 f 是环中一个边，那么两个点集一定会在某个地方再次连在一起，否则这就不会是一个圈，比如 e ）



于是，把 e 加入 $T - f$ 中，就重新把 A, B 连起来了，因此

$$T' = T - f + e$$

仍是一棵生成树。又因为 f 是环 C 上权重最大的边，所以

$$w(e) \leq w(f)$$

因此， f 不可能属于任何 MST。证毕。

Kruskal 正确性证明：

Kruskal 的关键动作是：

当某条边 e 被跳过时，是因为把它加入当前边集会形成一个环。

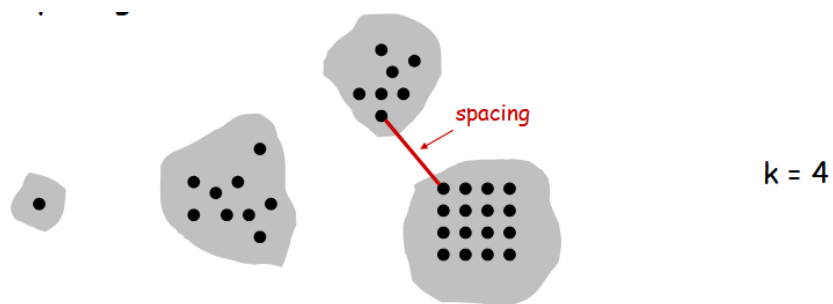
而在这个环里：

- 其他边都是之前已经被选过的边；
- 因为 Kruskal 按从小到大顺序扫描，故这些边的权重都**不大于** e ；
- 若边权互异，则都**严格小于** e 。

所以， e 是这个环上的**最重边**。也就是说，Kruskal 每次跳过一条边，跳过的都是不属于 MST 的边。

上面的 Prim 和 Kruskal 都建立在**边权各异**的情况下，但是有时我们也会遇到**边权相同**的情况，此时我们就可以采用字典序（or 其他编号），采取编号序。

Cluster 聚类、簇



我们把两个簇中任意两点间的最短的距离称为两个簇的距离，现在要研究的问题就是如何使这个最小距离最大（有段联想优化理论中的最小最大化问题）。

所以这个题要注意到的地方有两点：

- ①距离一定要是**最短**的那个连线
- ②这个连线一定要**尽可能大**

Single-link k -clustering 的做法是：

初始时每个点单独成簇，每次选择当前属于不同簇且**距离最近的一对点(有没有发现和找最小权值边一样)**，将其连接并合并为一个簇，重复 $n - k$ 次，直到剩下 k 个簇。

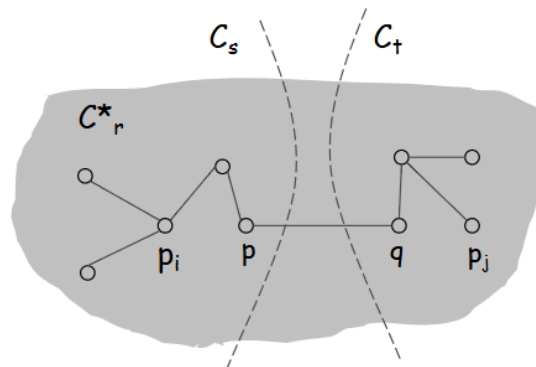
该过程本质上**等价于 Kruskal 算法在还剩 k 个连通分量时停止。**

进一步地，也等价于**先求最小生成树，再删除其中权重最大的 $k - 1$ 条边**，从而得到 k 个连通分量作为 k 个簇。

从直观上理解，删去的边也是曾经用 *Kruskal* 选出来的边，所以他们一定是两个点集间最小的距离，①得到了保证；而我们删去的是最后加上的几条边，*Kruskal* 最后选区的几条边，也就是最大的那几条边，由此②得到了保证。

正确性证明：

用到了非常巧妙地反证法：



先设：

- C^* ：删掉 MST 最大 $k - 1$ 条边得到的聚类
- d^* ：这些被删边里最小的那个权值
| 也就是 C^* 的 spacing

假设还有**另一个聚类 C** ，它比 C^* 更好，即：

$$\text{spacing}(C) > d^*$$

为什么会产生矛盾？

因为 $C \neq C^*$ ，所以一定存在两个点：

- 在 C^* 里属于同一个簇
- 但在 C 里被分到了不同簇

这两个点在 MST 里有唯一一条路径。既然它们在 C 里分属不同簇，那么这条路径上一定有一条边 (p, q) 跨过了 C 的簇边界。

而这条边 (p, q) 还留在 C^* 的某个连通块里，说明它**没有被删掉**，所以它的边权不会超过 d^* ：

$$w(p, q) \leq d^*$$

但 (p, q) 又是一条跨越 C 不同簇的边，**所以 C 的最小间距一定不超过它**：

$$\text{spacing}(C) \leq w(p, q) \leq d^*$$

这就和前面假设的

$$\text{spacing}(C) > d^*$$

矛盾。

这个反证法其实用到的是**结构化证明思想**，也就是给算法找一个**上界**，很显然，通过上面的证明，我们发现这个上界就是 d^* ，我们要想办法证明：任何别的聚类都不可能把 *spacing* 做到超过 d^* 。

类似的证明思路还有 *Dijkstra* 正确性的证明，也是通过**构造一个界**，只不过构造的是下界。

更加有意思的是，*Dijkstra* 和本题的证明，都是以两个点集的连接处为突破点。这是因为**图算法中，最常见的场景就是选择路径**，本章与 *Dijkstra* 都是在做贪心路径选择，所以我们才要将重点放在连接处（也就是做选择处）。只要我们证明出来每一步贪心都是那个界，就可以归纳地得出贪心最优。

其实每一步贪心都是那个界，也体现了 *stays ahead* 思想。